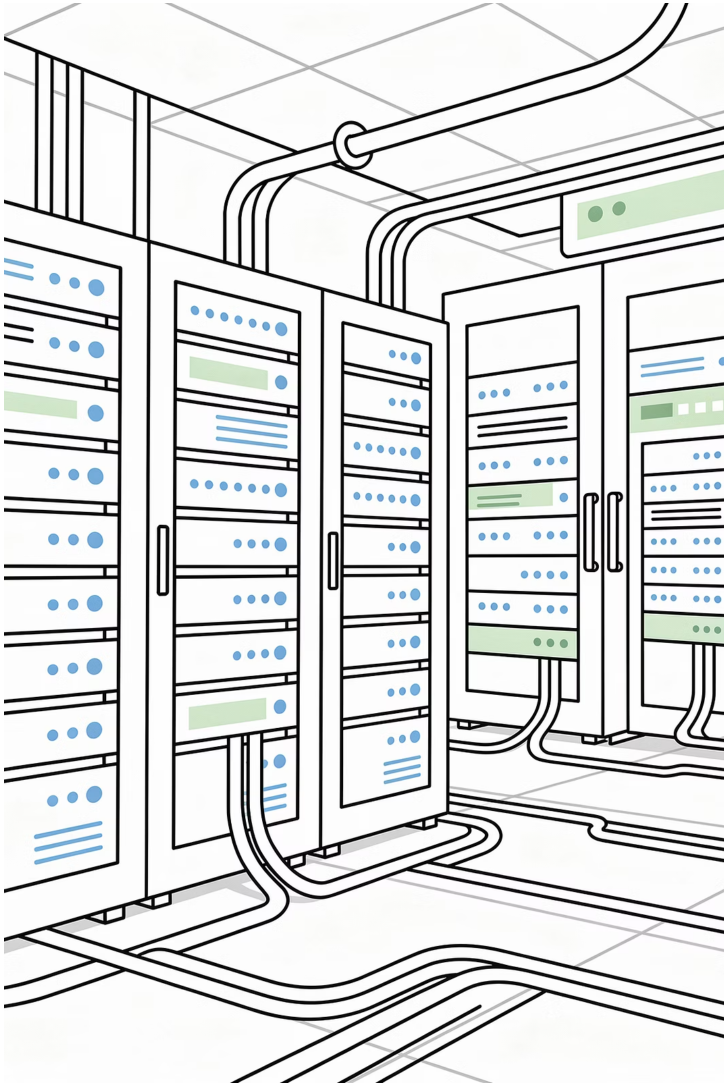
Өгөгдлийн сангийн удирдлага

F.ITM237

Мэдээллийн технологийн тэнхим

С.Өлзийбаяр (Ph.D)



Агуулга

- Relational Set operators (Union, Intersect, Minus)
- Joins (Inner, Outer, Cross Joins)
- Subqueries and Nested Queries
- Views, Indexes, and Sequences
- Transactions and Concurrency Control

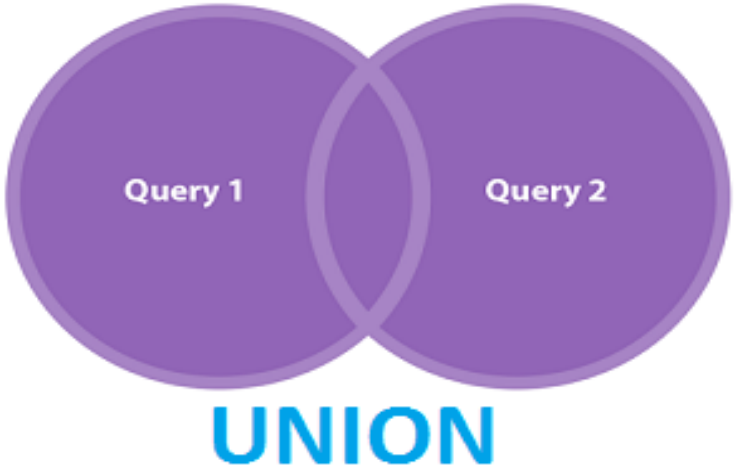
Relational Set Operators (SQL)

- **Олонлогийн операторууд** нь SQL-ийн өгөгдлийг хүснэгтүүдийг нэгтгэхэд ашигладаг операторууд юм.
- Эдгээр нь **UNION**, **UNION ALL**, **INTERSECT**, **MINUS** операторууд байдаг. Эдгээр операторуудын тусламжтайгаар нэгээс олон хүснэгтээс өгөгдлийг нэгтгэх, харьцуулах, ялгах боломжтой.

Relational Set Operators (SQL)

UNION оператор нь хоёр SQL асуулгын үр дүнг нэгтгэж, давхцсан мөрүүдийг нэг удаа буцаадаг.

```
SELECT name FROM employees
UNION
SELECT name FROM customers;
```



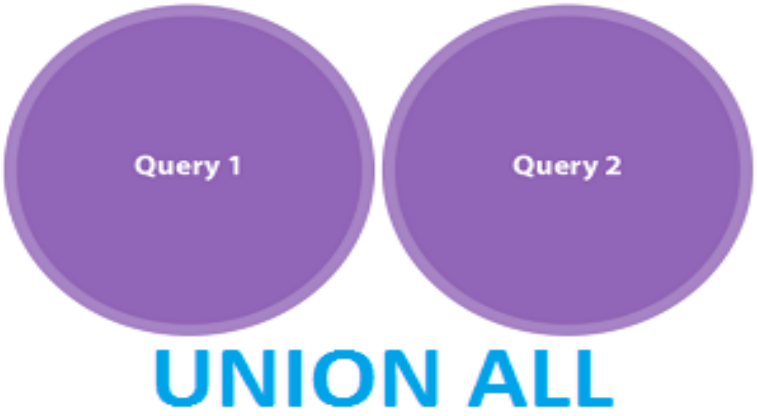
*Энэ нь **employees** ба **customers** хүснэгтүүдийн нийтлэг нэрсийг буцаана, давхардсан мөрийг нэг удаа харуулна.*

•**UNION-ийн онцлог**: Давхардсан мөрүүдийг автоматаар хасаж, зөвхөн дахин давтагдаагүй мөрүүдийг харуулдаг.

Relational Set Operators (SQL)

UNION ALL оператор нь хоёр асуулгын бүх үр дүнг буцааж, давхардсан мөрүүдийг хасахгүйгээр харуулдаг.

```
SELECT name FROM employees
UNION ALL
SELECT name FROM customers;
```



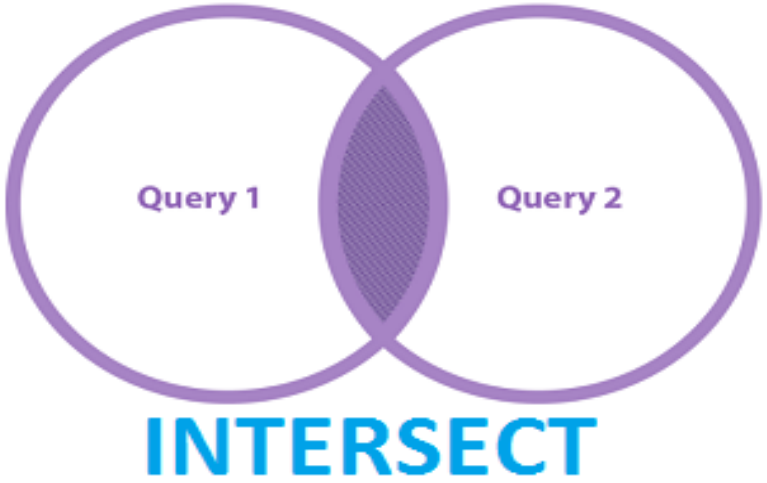
Энэ нь **employees** ба **customers** хүснэгтүүдийн бүх мөрийг, давхардсан байлаа ч харуулна.

•**UNION ALL-ийн онцлог**: Давхардсан мөрүүдийг оруулдаг тул асуулгын үр дүн нь хурдан гарч ирэх боломжтой.

Relational Set Operators (SQL)

INTERSECT оператор нь хоёр асуулгын нийтлэг мөрүүдийг буцаана. Зөвхөн хоёуланд нь давхцаж буй өгөгдлийг харуулна.

```
SELECT name FROM employees
INTERSECT
SELECT name FROM customers;
```



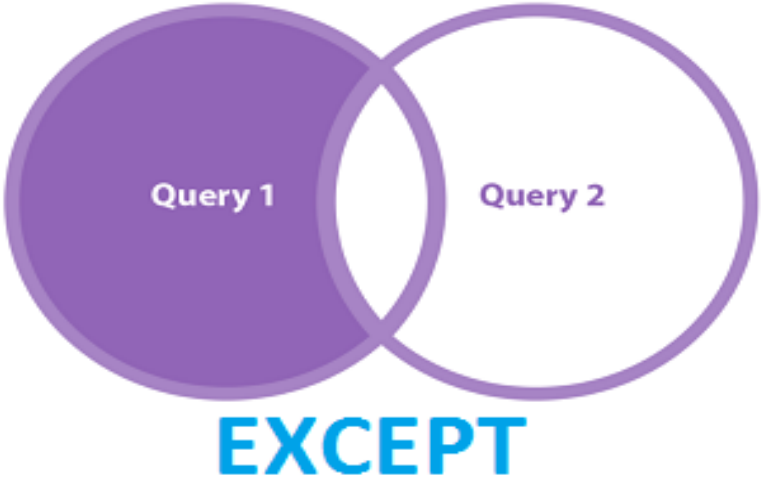
Энэ нь **employees** болон **customers** хүснэгтүүдийн нийтлэг нэрсийг буцаана.

INTERSECT-ийн онцлог: Зөвхөн хоёр хүснэгтийн дундах нийтлэг өгөгдлийг харуулдаг

Relational Set Operators (SQL)

MINUS (EXCEPT) оператор нь эхний асуулгын үр дүнгээс хоёр дахь асуулгын мөрүүдийг хасаж үлдсэн мөрүүдийг буцаана.

```
SELECT name FROM employees
MINUS
SELECT name FROM customers;
```



Энэ нь **employees** хүснэгтэд байгаа, харин **customers** хүснэгтэд байхгүй нэрсийг буцаана.

MINUS-ийн онцлог: Нэг хүснэгтээс нөгөөд байхгүй өгөгдлийг гаргаж харуулдаг.

Relational Set Operators (SQL)

- Өгөгдлийн сангийн бүтэц болон харилцан хамааралтай олон өгөгдлийг нэгтгэх шаардлагатай үед чухал.
- Тодорхой нөхцөлд хэрэглэгддэг ба, жишээ нь: **UNION** ба **UNION ALL** нь ихэвчлэн хүснэгтүүдийг нэгтгэхэд, харин **INTERSECT** ба **MINUS** нь өгөгдлийг харьцуулахад ашиглагддаг.

Relational Set Operators (SQL)

```
SELECT name FROM employees
INTERSECT
SELECT name FROM customers
MINUS
SELECT name FROM former_employees;
```

Энэ жишээ нь **employees** болон **customers** хүснэгтүүдийн нийтлэг өгөгдлийг буцаах ба, үүнийг **former_employees** хүснэгтэд байхгүй гэдгийг шалгаж байна.

Эдгээр олонлогийн операторуудыг зөв зохистой хэрэглэж сурах нь SQL асуулгуудыг илүү үр дүнтэй болгох болно.

Joins (холбоосууд)

- **Joins (Inner, Outer, Cross Joins)** нь SQL-д олон хүснэгтээс өгөгдлийг нэгтгэх аргачлалууд юм. Хүснэгтүүдийн хоорондын харилцааг тодорхойлж, өгөгдлийг нэгтгэн нэг хүснэгтээс үр дүн гаргах боломжийг олгодог.

Join-ийн төрлүүд

INNER JOIN:

Хоёр хүснэгтэд давхцаж буй мөрүүдийг буцаана.

LEFT JOIN:

Зүүн хүснэгтээс бүх мөрүүдийг буцаах ба баруун хүснэгттэй таарсан мөрүүдийг оруулна.

RIGHT JOIN:

Баруун хүснэгтийн бүх мөрүүдийг буцаана, зүүн хүснэгттэй таарсан мөрүүдийг оруулна.

FULL OUTER JOIN:

Хоёр хүснэгтийн бүх мөрүүдийг буцаана.

CROSS JOIN:

Бүх мөрүүдийг хослуулан үр дүнг гаргана.

INNER JOIN

INNER JOIN нь хоёр хүснэгтэд орших нийтлэг өгөгдлийг нэгтгэн буцаадаг. Энэ нь хоёр хүснэгтийн хооронд давхцаж буй мөрүүдийг нэгтгэхэд ашиглагдана.

```
SELECT employees.name, departments.department_name
FROM employees
INNER JOIN departments
ON employees.department_id = departments.id;
```

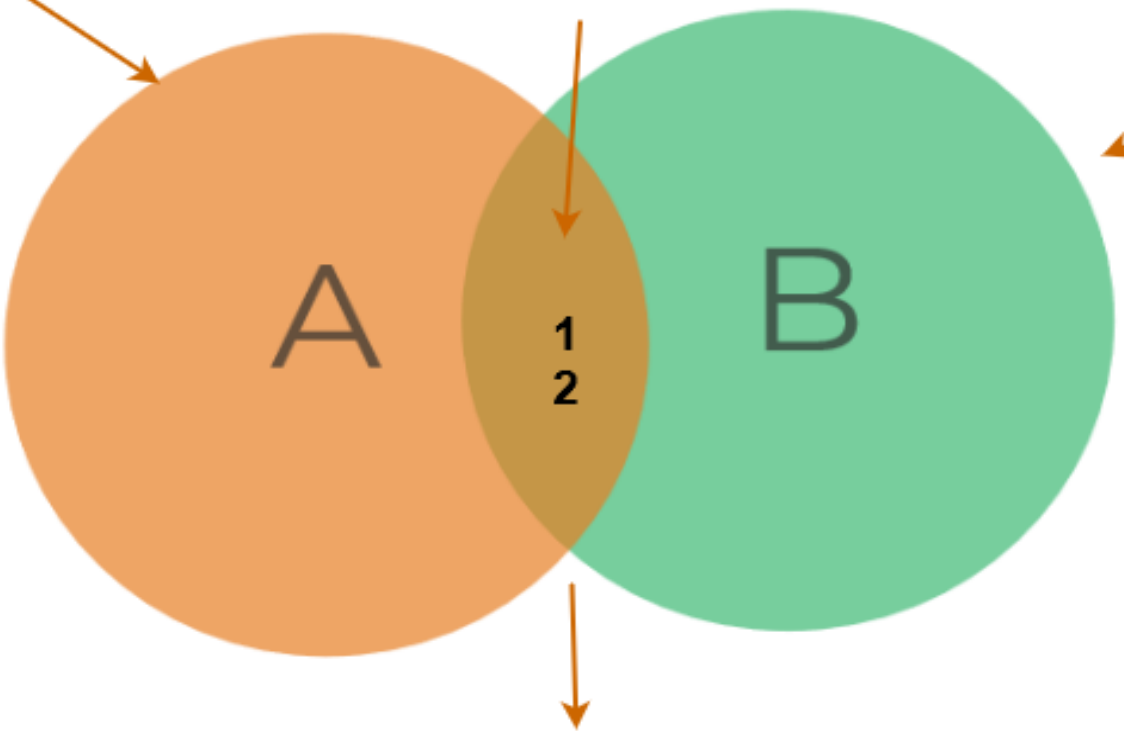
Энэ жишээнд **employees** болон **departments** хүснэгтүүдийн нийтлэг мөрүүдийг, **department_id** болон **id** багануудын хоорондын холбоогоор нэгтгэн харуулж байна.

•**Онцлог**: *Зөвхөн хоёр хүснэгтийн хооронд давхцаж буй өгөгдлийг буцаана. Хэрэв мөрүүд давхцахгүй бол ямар ч өгөгдөл буцахгүй.*

EmpID	EmpName
1	Rajendra
2	Kusum
3	Akshita

Inner Join

ID	City
1	Jaipur
2	Delhi
7	Raipur
8	Bangalore

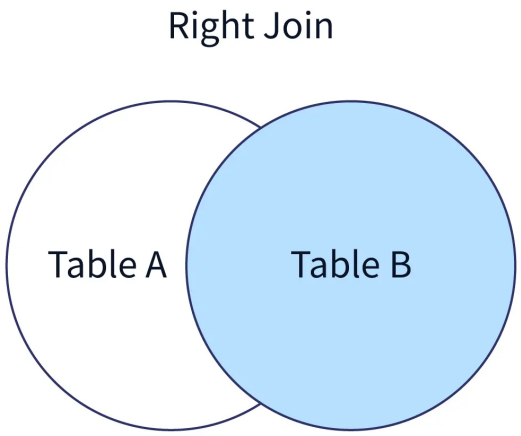


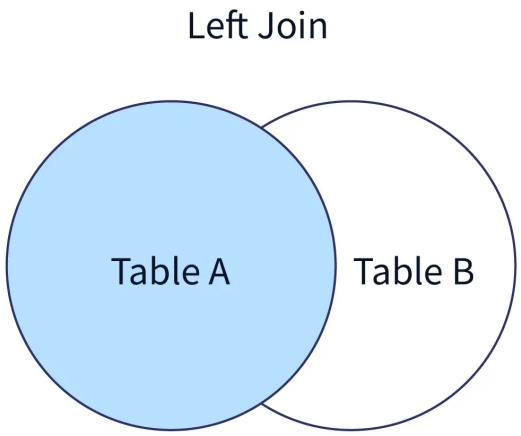

```
Select EmpName, City from employees e
Inner Join address a
on e.empid=a.id
```

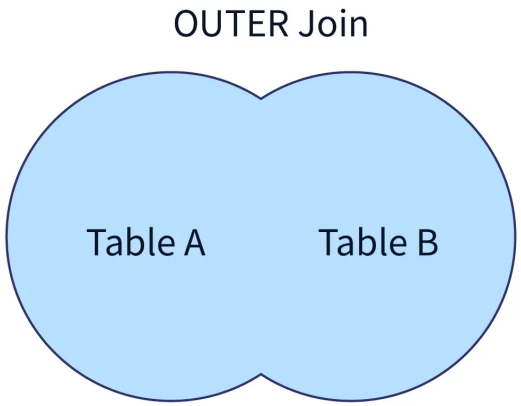
EmpName	City
Rajendra	Jaipur
Kusum	Delhi

OUTER JOIN (Гадаад холбоос)

OUTER JOIN нь хоёр хүснэгтийн хооронд давхцах өгөгдлөөс гадна, нэг хүснэгтийн бүх өгөгдлийг буцаах боломжтой. **OUTER JOIN** нь **LEFT JOIN**, **RIGHT JOIN**, **FULL OUTER JOIN** гэсэн гурван төрлөөр хэрэглэгддэг.

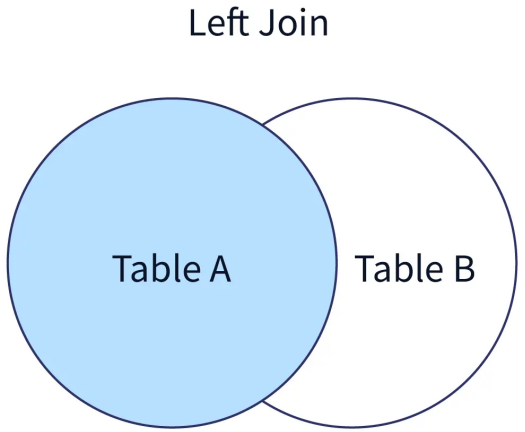






LEFT JOIN

LEFT JOIN нь зүүн талын хүснэгтийн бүх мөрүүдийг буцааж, баруун хүснэгттэй таарсан мөрүүдийг оруулдаг. Хэрэв баруун хүснэгттэй тааараагүй бол NULL утга оруулдаг.

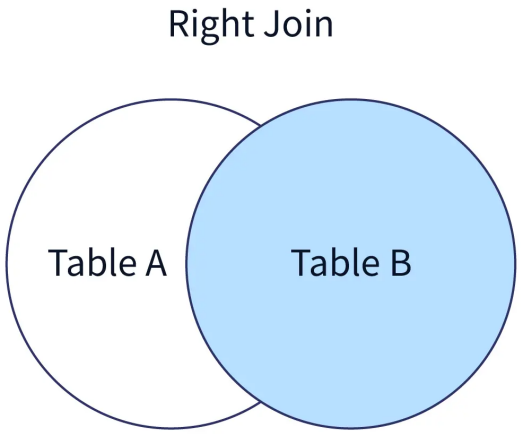



```
SELECT employees.name, departments.department_name
FROM employees
LEFT JOIN departments
ON employees.department_id = departments.id;
```

Энэ жишээнд **employees** хүснэгтийн бүх мөрүүдийг харуулж, **departments** хүснэгтээс таарсан өгөгдлийг буцаана.

RIGHT JOIN

RIGHT JOIN нь баруун талын хүснэгтийн бүх мөрүүдийг буцааж, зүүн талын хүснэгттэй таарсан өгөгдлийг оруулдаг. Хэрэв зүүн талын хүснэгттэй таарахгүй бол NULL утга оруулдаг.

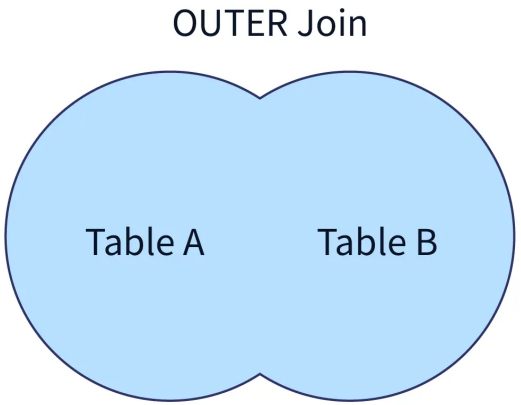



```
SELECT employees.name, departments.department_name
FROM employees
RIGHT JOIN departments
ON employees.department_id = departments.id;
```

Энэ жишээнд **departments** хүснэгтийн бүх мөрүүдийг харуулж, **employees** хүснэгтээс таарсан өгөгдлийг буцаана.

FULL OUTER JOIN

FULL OUTER JOIN нь хоёр хүснэгтийн бүх мөрүүдийг буцааж, таарсан мөрүүдийг нэгтгэн харуулдаг. Хэрэв мөрүүд таарахгүй бол NULL утга оруулна.




```
SELECT employees.name, departments.department_name
FROM employees
FULL OUTER JOIN departments
ON employees.department_id = departments.id;
```

Энэ жишээнд **employees** болон **departments** хүснэгтүүдийн бүх мөрүүдийг буцааж, таарсан өгөгдлийг нэгтгэн харуулна.

CROSS JOIN

CROSS JOIN нь хоёр хүснэгтийн бүх мөрүүдийг хослуулж, өгөгдөл үүсгэхэд ашиглагддаг. Энэ нь хоёр хүснэгтийн мөрүүдийг хооронд нь зөрүүлж нийлүүлэн үр дүн гаргана. **CROSS JOIN** нь картезиан үржвэр гэж нэрлэгддэг.

```
SELECT employees.name, departments.department_name
FROM employees
CROSS JOIN departments
```

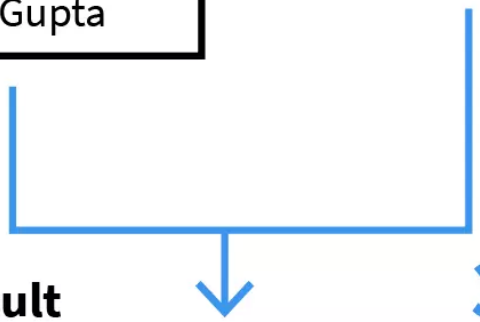
- Энэ жишээнд **employees** хүснэгтийн бүх мөрүүдийг **departments** хүснэгтийн бүх мөрүүдтэй хослуулж харуулна. Хэрэв **employees** хүснэгтэд 5 мөр, **departments** хүснэгтэд 3 мөр байвал нийт 15 мөр үр дүн гарна.
- **Онцлог**: Хүснэгтүүдийн мөр бүрийг нөгөө хүснэгтийн мөр бүртэй хослуулан өгөгдөл гаргадаг.

Table : Customers

ID	Name
1	Raj Mehta
2	Sanjay Mishra
3	Aditi Gupta

Table_Name : Shopping_Details

ID	Item_Name
2	Chips
3	Chocolate



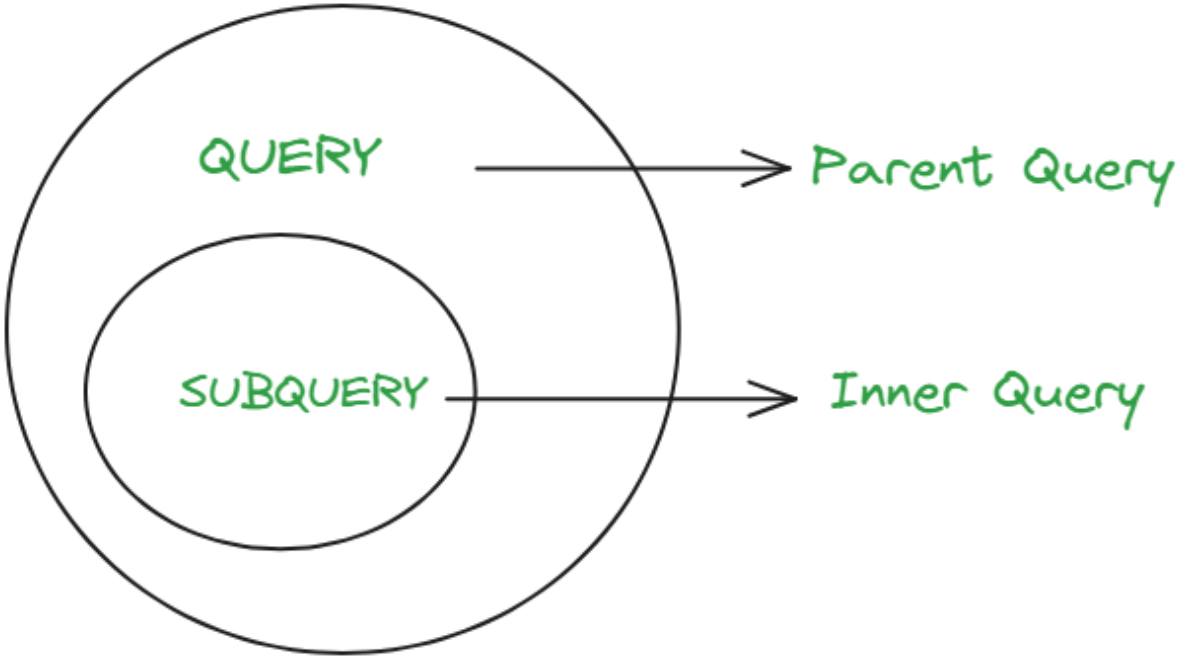
3 x 2 = 6 rows

Table : Result

Customers CustID	Name	Shopping_details.CustID	Item_Name
1	Raj Mehta	2	Chips
1	Raj Mehta	3	Chocolate
2	Sanjay Mishra	2	Chips
2	Sanjay Mishra	3	Chocolate
3	Aditi Gupta	2	Chips
3	Aditi Gupta	3	Chocolate

Subqueries ба Nested Queries

(Subqueries) ба (Nested Queries) нь SQL-д нэг асуулгын дотор өөр нэг асуулгыг оруулж, илүү нарийн мэдээллийг гаргаж авах боломжийг олгодог аргачлал юм. Энэ нь өгөгдлийг илүү динамик байдлаар боловсруулахад ашиглагддаг.



Subqueries ба Nested Queries

- Дэд асуулгын ойлголт: Дэд асуулга нь үндсэн асуулгад зориулан өгөгдлийг буцаах.

- Нийтлэг хэрэглээ: SELECT, INSERT, UPDATE, DELETE командууд дотор дэд асуулгуудыг бичих.

Subqueries ба Nested Queries

Дэд асуулга (Subquery) гэж юу вэ?

Дэд асуулга нь нэг SQL асуулгын доторх өөр нэг SQL асуулга бөгөөд үндсэн асуулгадаа ашиглах утгыг олж өгдөг. Үүнийг хаалт (()) ашиглан бичдэг.

Дэд асуулгын онцлог:

•Дэд асуулгыг **SELECT**, **INSERT**, **UPDATE**, **DELETE** командыг дотор нь ашиглаж болно.

•Дэд асуулга нь ихэвчлэн нэг эсвэл хэд хэдэн утгыг буцааж өгдөг.

```
SELECT name, salary FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
```

*Энэ жишээнд **employees** хүснэгтээс цалингийн дундаж утгаас их цалинтай ажилчдыг буцаана. Дэд асуулга нь цалингийн дундаж утгыг олж өгдөг.*

Subqueries ба Nested Queries

Нэг мөрт дэд асуулга нь зөвхөн нэг утгыг буцаадаг дэд асуулга юм. Үүнийг ихэвчлэн **=, >, <, >=, <=** зэрэг операторуудтай хамт хэрэглэдэг.

```
SELECT name, department_id FROM employees
WHERE department_id =
(SELECT department_id FROM departments WHERE
department_name = 'HR');
```

Энэ жишээнд **departments** хүснэгтээс 'HR' гэсэн нэртэй хэлтсийн ID-г олоод, түүнийг үндсэн асуулгад хэрэглэж байна.

Subqueries ба Nested Queries

Олон мөрт дэд асуулга нь хэд хэдэн утгыг буцаах боломжтой бөгөөд **IN**, **ANY**, **ALL** зэрэг операторуудыг ашигладаг.

```
SELECT name, department_id
FROM employees
WHERE department_id IN (SELECT department_id FROM
departments WHERE location = 'Ulaanbaatar');
```

Энэ жишээнд **departments** хүснэгтээс **Ulaanbaatar** хотод байрладаг бүх хэлтсийн ID-үүдийг буцааж, үндсэн асуулгад ашиглаж байна.

Subqueries ба Nested Queries

Nested Query нь олон давхар дэд асуулга агуулсан SQL асуулга юм. Нэг асуулгын дотор өөр асуулга, тэр асуулгын дотор дахин дэд асуулга бичиж, илүү нарийн өгөгдлийг олж авахад ашигладаг.

```
SELECT name
FROM employees
WHERE department_id = (SELECT department_id
                       FROM departments
                       WHERE location_id = (SELECT location_id
                                            FROM locations
                                            WHERE city = 'Ulaanbaatar'));
```

- Энэ жишээнд хамгийн доорхи **locations** хүснэгтээс **Ulaanbaatar** хотын ID-г олж, дараагийн шатны дэд асуулгад ашиглаж байна. Үүний дараа олдсон ID-г ашиглан **departments** хүснэгтээс хэлтсийн ID-г гаргаж авна. Эцэст нь тэр хэлтсийн ажилчдыг үндсэн асуулга харуулна.

Subqueries ба Nested Queries

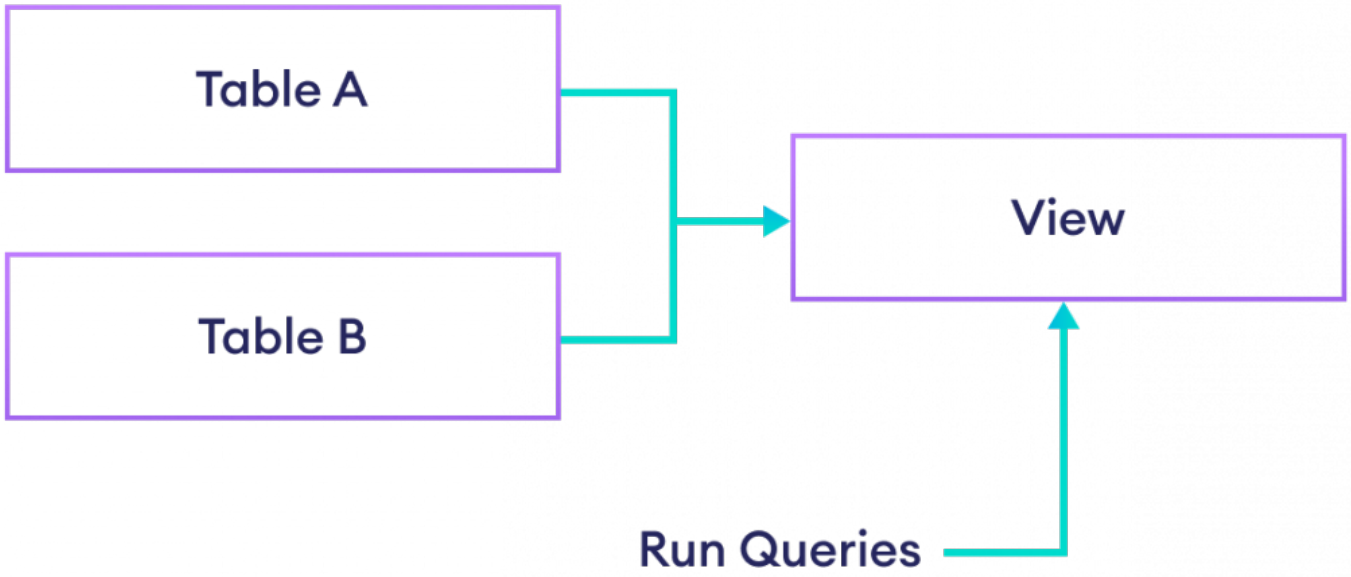
```
INSERT INTO high_salary_employees (name, salary)
SELECT name, salary
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
```

```
UPDATE employees
SET salary = salary * 1.10
WHERE department_id = (SELECT department_id FROM
departments WHERE department_name = 'HR');
```

Views, Indexes, and Sequences

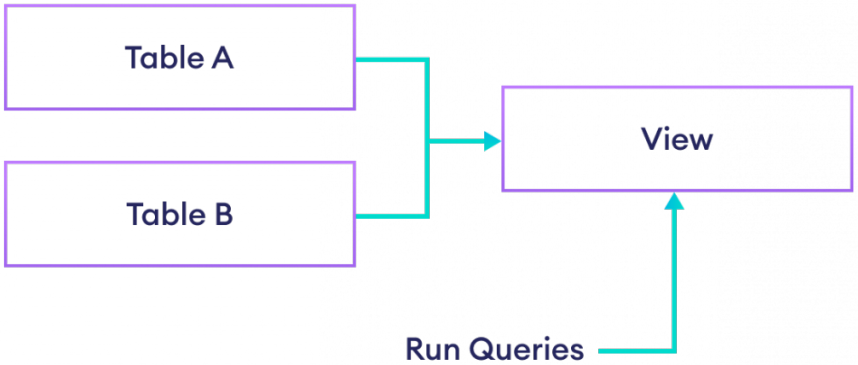
•**Views, Indexes, Sequences** нь SQL өгөгдлийн санг оновчтой болгох, өгөгдөлтэй ажиллах үйл ажиллагааг хялбарчлахад ашиглагддаг хэрэгслүүд юм.

•Эдгээр нь өгөгдлийн сангийн бүтцийн гүйцэтгэлийг сайжруулж, өгөгдөлтэй харьцах илүү хялбар арга замыг хэрэглэгчдэд олгодог.



Views, Indexes, and Sequences

•**View** буюу **Харагдац** нь SQL-ийн виртуал хүснэгт юм. Энэ нь өгөгдлийг хадгалаагүй ч SQL асуулгыг түр хадгалж, хүссэн үедээ хэрэглэж болох хэлбэрээр үүсгэдэг. Үндсэн хүснэгтүүд дээр тулгуурлан өгөгдлийг шүүж, хялбаршуулсан хэлбэрээр үзүүлэх боломжийг олгодог.




```
CREATE VIEW employee_view AS
SELECT name, salary, department_id
FROM employees
WHERE salary > 5000;
```

Энэ жишээнд **employee_view** харагдац үүсгэж, зөвхөн цалингийн хэмжээ 5000-аас дээш ажилчдын мэдээллийг харуулж байна.

```
SELECT * FROM employee_view;
```

Views, Indexes, and Sequences

employee_view харагдацыг ашиглан ажилчдын мэдээллийг харах боломжтой.

View ашиглах:

```
SELECT * FROM employee_view;
```

View-ийн ашиг тус:

- Хүснэгтүүдийн өгөгдлийг олон удаа давтагдахгүйгээр хялбаршуулсан байдлаар харах.
- Нууцлалын шаардлагад нийцүүлэн зарим өгөгдлийг харуулахгүй байх боломж.
- Нарийн төвөгтэй SQL асуулгуудыг хялбар хэлбэрт оруулах.

Views, Indexes, and Sequences

- **Индекс** нь өгөгдлийн санд өгөгдлийг хурдан хайхад ашиглагддаг бүтцийн элемент юм.
- Хүснэгт дэх өгөгдөл рүү хандах хурдыг нэмэгдүүлэх үүрэгтэй.
- Индекс нь өгөгдлийн баганын утгууд дээр үндэслэн үүсгэгддэг ба хүснэгтийн мөрүүдийг хурдан олох боломжийг олгодог.

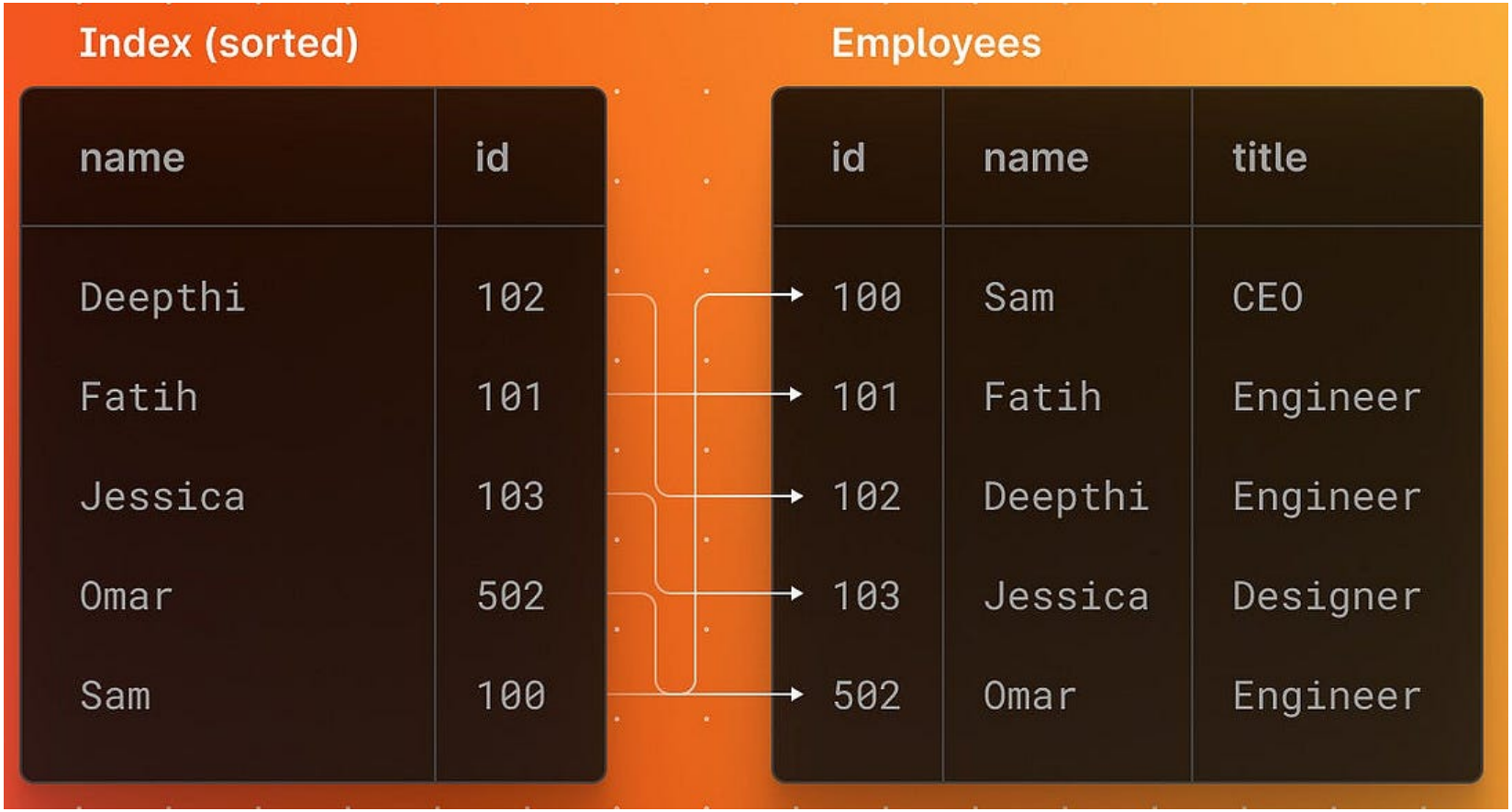
Index үүсгэх:

```
CREATE INDEX idx_employee_name ON employees(name);
```

- *Энэ жишээнд **employees** хүснэгтийн **name** баганад зориулж индекс үүсгэж байна.*

Views, Indexes, and Sequences

```
CREATE INDEX idx_employee_name ON employees(name);
```



Views, Indexes, and Sequences

Индексийн ашиг тус:

- Асуулгын гүйцэтгэлийг сайжруулж, өгөгдлийн санд өгөгдлийг хурдан хайх боломжийг олгоно.
- Хэрэглэгчийн асуулгын цаг хугацааг хэмнэнэ.

Индексийн төрлүүд:

- **Нэг баганын индекс**: Нэг баганад үндэслэн үүсгэгддэг. (Жишээ нь дээрх индекс)
- **Нийлмэл индекс (Composite Index)**: Олон баганын утгуудыг ашиглан үүсгэгддэг.

```
CREATE INDEX idx_employee_dept
ON employees(department_id, name);
```

- Энэ жишээнд **department_id** болон **name** баганууудын нийлмэл индексийг үүсгэж байна.

Views, Indexes, and Sequences

Sequence буюу **Дараалал** нь SQL-д автоматаар өөрчлөгдөж буй дугаарыг үүсгэхэд ашиглагддаг механизм юм.

- Энэ нь ихэвчлэн анхдагч түлхүүр (Primary Key) эсвэл тодорхой дугаарлалт шаардлагатай үед хэрэглэгддэг.

```
CREATE SEQUENCE employee_seq
START WITH 1
INCREMENT BY 1;
```

- *Энэ жишээнд **employee_seq** гэдэг нэртэй дарааллыг үүсгэж, эхний утга нь 1, цаашид 1-ээр нэмэгдэх байдлаар үүсгэж байна.*

Views, Indexes, and Sequences

```
CREATE SEQUENCE employee_seq
START WITH 1
INCREMENT BY 1;
```

Sequence-ийн ашиг тус:

- Анхдагч түлхүүрийн утгыг автоматаар үүсгэх.
- Тодорхой дараалал бүхий утгуудыг системд автоматаар оруулдаг тул гар аргаар дугаар оруулах шаардлагагүй болдог.

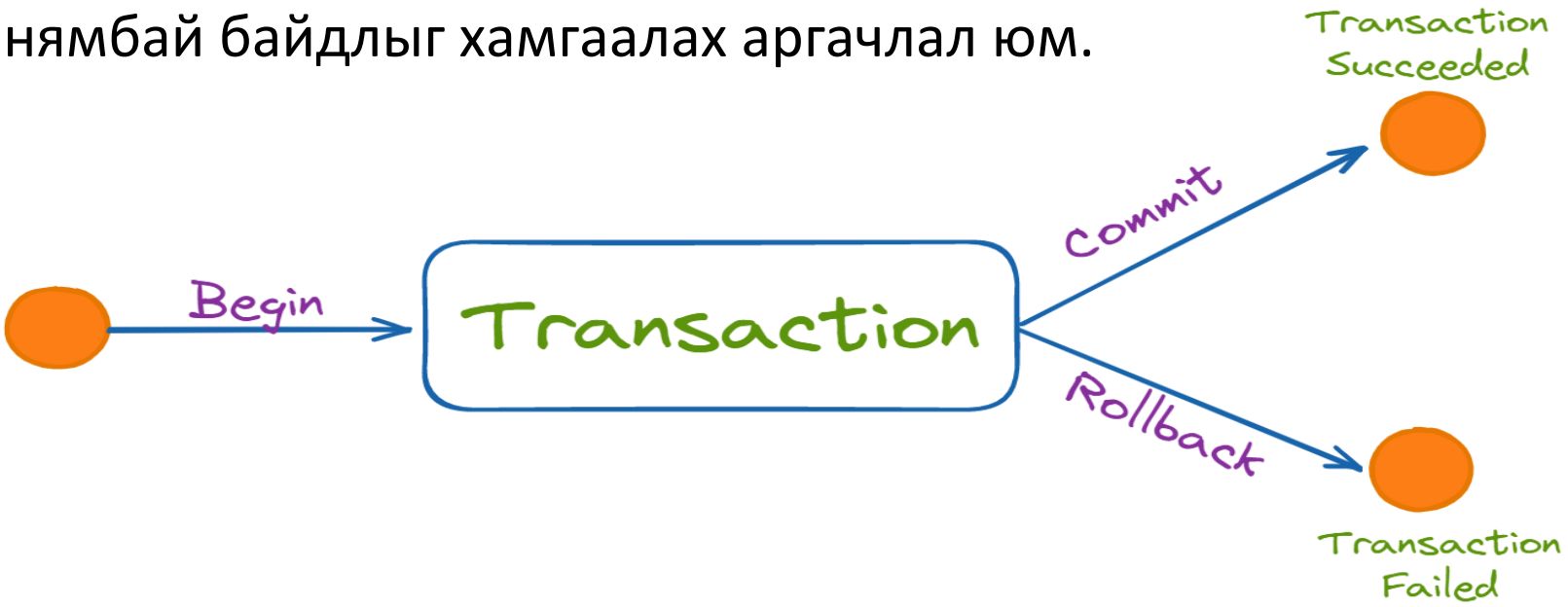
Sequence ашиглах:

```
INSERT INTO employees (id, name, salary)
VALUES (employee_seq.NEXTVAL, 'Bat', 6000);
```

- **employee_seq** дарааллын дараагийн утгыг **id** баганад автоматаар оруулж өгч байна.

Transactions and Concurrency Control

- **Гүйлгээ (Transactions) болон Зэрэгцээ удирдлага (Concurrency Control)** нь SQL өгөгдлийн санд өгөгдлийн бүрэн бүтэн байдал, найдвартай ажиллагааг хангах чухал механизм юм.
- Энэ нь олон хэрэглэгч нэгэн зэрэг өгөгдөлтэй ажиллах үед өгөгдлийн зөрчилдөөнөөс сэргийлэх, өгөгдлийн нягт нямбай байдлыг хамгаалах аргачлал юм.



Transactions and Concurrency Control

Гүйлгээ (Transactions)

Гүйлгээ нь SQL өгөгдлийн санд хийж буй олон үйлдлийг нэгэн зэрэг гүйцэтгэх процессыг илэрхийлдэг.

- Нэг гүйлгээ нь хэд хэдэн SQL командуудыг хамарч, эдгээр командууд бүгд амжилттай дууссан тохиолдолд өгөгдлийн санд хийгдсэн өөрчлөлтүүдийг байнгын болгодог.

ACID шинж чанар:

Гүйлгээг найдвартай байлгахын тулд **ACID** шинж чанаруудыг дагаж мөрддөг.

•**A**: ***Atomicity*** – Гүйлгээний бүх үйлдэл амжилттай хийгдэх эсвэл ямар ч өөрчлөлт хийгдэхгүй байх ёстой.

•**C**: ***Consistency*** – Гүйлгээ дууссаны дараа өгөгдлийн сангийн төлөв зохих дүрмүүдийг зөрчихгүй байх ёстой.

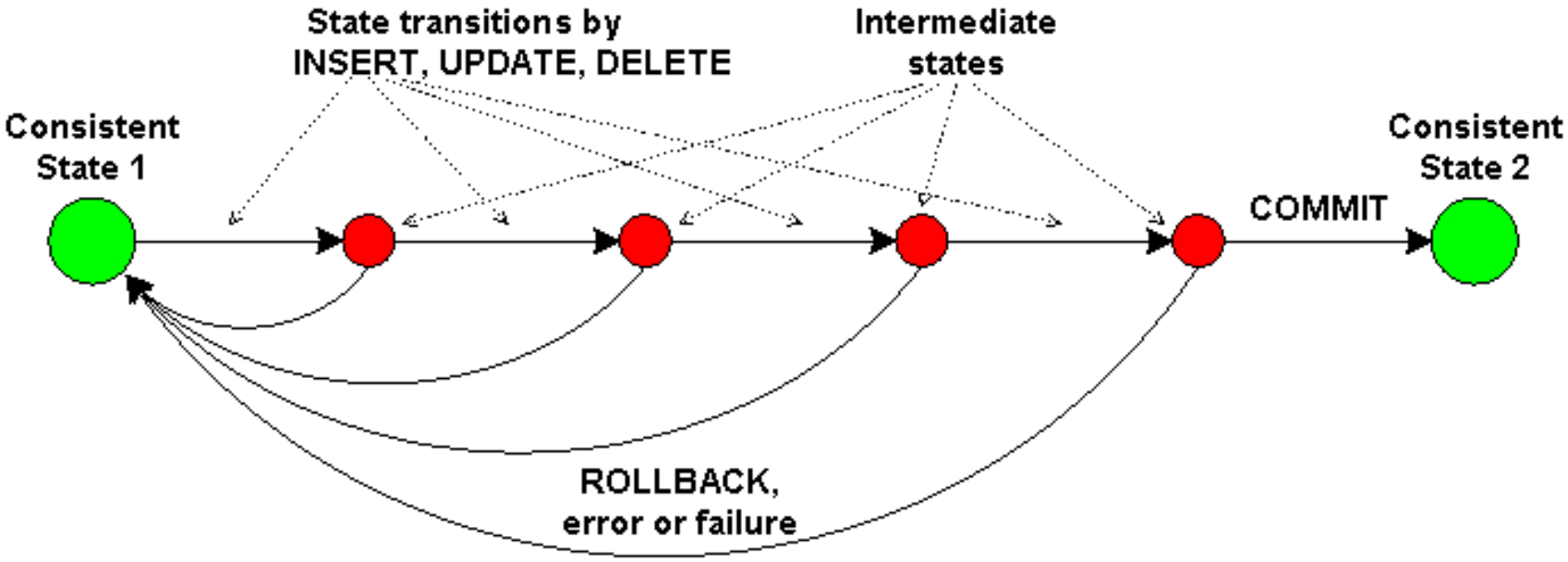
•**I**: ***Isolation***– Нэг гүйлгээний явцад бусад гүйлгээнээс хараат бус байх ёстой.

•**D**: ***Durability*** – Гүйлгээ дууссаны дараа хийсэн өөрчлөлтүүд хадгалагдаж үлдэх ёстой.

Transactions and Concurrency Control

Гүйлгээний командууд:

- **BEGIN TRANSACTION**: Гүйлгээг эхлүүлэх.
- **COMMIT**: Гүйлгээг амжилттай дуусгаж, өгөгдлийн санд хийсэн өөрчлөлтүүдийг байнгын болгох.
- **ROLLBACK**: Гүйлгээг цуцлаж, хийсэн өөрчлөлтүүдийг буцаах.
- **SAVEPOINT**: Гүйлгээний дундуур тэмдэглэл тавьж, тухайн цэгээс хойш буцаах боломжтой болгох.



COMMIT ба ROLLBACK хосолсон жишээ

```
BEGIN TRANSACTION;
UPDATE accounts
SET balance = balance - 100
WHERE account_number = 123;
IF @@ERROR <> 0
BEGIN
ROLLBACK;
  PRINT 'Гүйлгээ амжилтгүй боллоо.';
  RETURN;
END;
UPDATE accounts
SET balance = balance + 100
WHERE account_number = 456;
IF @@ERROR <> 0
BEGIN
ROLLBACK;
  PRINT 'Гүйлгээ амжилтгүй боллоо.';
  RETURN;
END;
COMMIT;
PRINT 'Гүйлгээ амжилттай дууслаа.';
```

Гүйлгээний тайлбар:

1.BEGIN TRANSACTION: Гүйлгээг эхлүүлнэ.

2.Эхний данснаас мөнгө хасах үйлдэл хийгдэнэ.

3.Хэрэв энэ үйлдэлд алдаа гарвал **ROLLBACK** ашиглан бүх үйлдлийг цуцална.

4.Алдаа байхгүй бол хоёр дахь дансанд мөнгө нэмнэ.

5.Дахин алдаа шалгагдана, хэрэв алдаа гарвал **ROLLBACK**.

6.Хэрэв бүх үйлдэл амжилттай болвол **COMMIT** ашиглан бүх өөрчлөлтийг баталгаажуулна.

SAVEPOINT ашиглах

```
BEGIN TRANSACTION;

UPDATE accounts
SET balance = balance - 100
WHERE account_number = 123;

SAVEPOINT save1;

UPDATE accounts
SET balance = balance + 100
WHERE account_number = 456;

ROLLBACK TO save1;

COMMIT;
```

Гүйлгээний удирдлагын командууд:

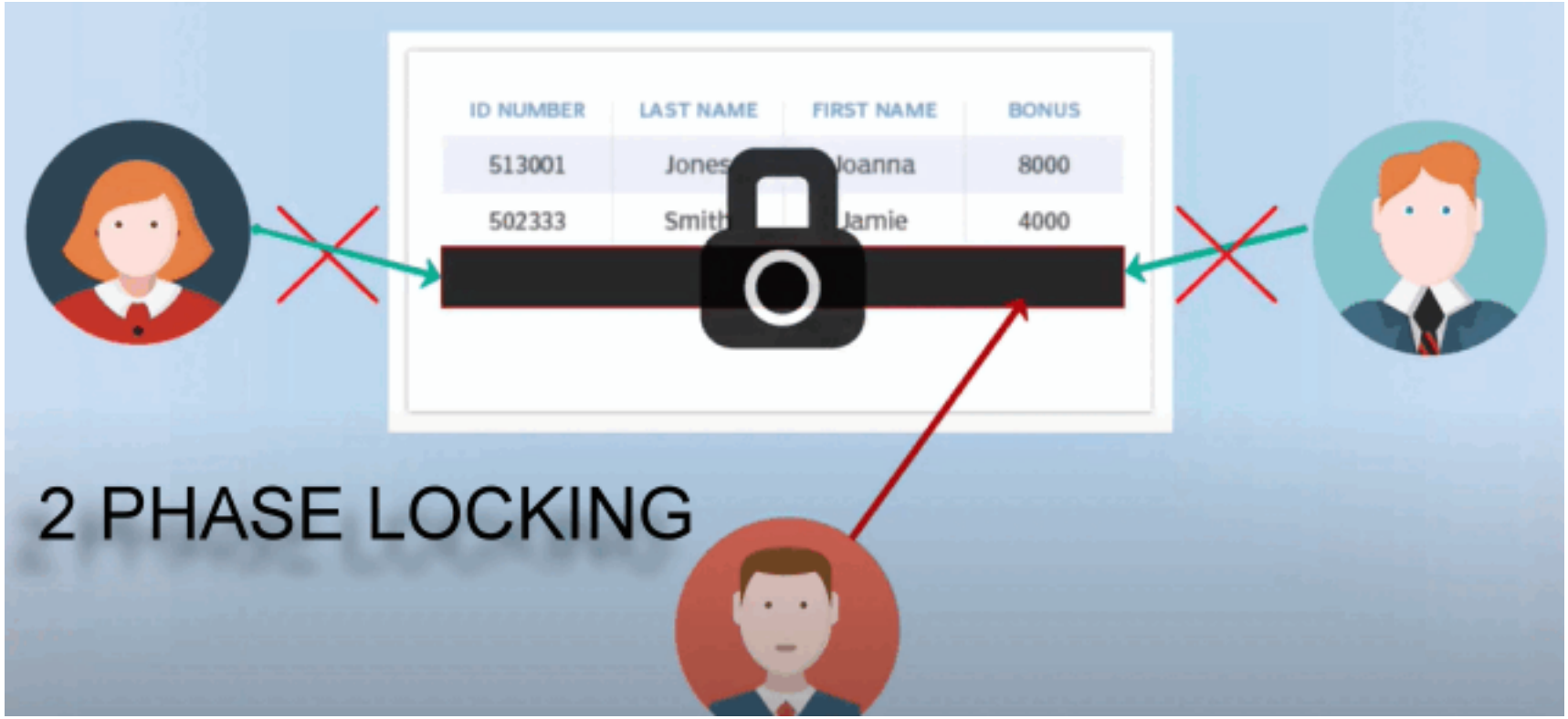
- **COMMIT**: Гүйлгээ дуусч, өгөгдөлд байнгын өөрчлөлт хийнэ.
- **ROLLBACK**: Гүйлгээг цуцалж, өгөгдөлд хийсэн өөрчлөлтийг буцаана.
- **SAVEPOINT**: Гүйлгээний тодорхой хэсэгт тэмдэглэл тавьж, шаардлагатай үед тухайн цэгээс хойш буцах боломжтой.

*Энэ жишээнд **SAVEPOINT** ашиглаж, тодорхой цэгээс гүйлгээг буцааж, зөвхөн эхний өөрчлөлтийг баталгаажуулж байна.*

Transactions and Concurrency Control

Concurrency Control:

Зэрэгцээ удирдлага нь олон хэрэглэгч нэгэн зэрэг өгөгдөлд хандах үед өгөгдлийн бүрэн бүтэн байдлыг хадгалах механизмууд юм. SQL өгөгдлийн сан нь зэрэгцээ гүйлгээний асуудлуудаас сэргийлэхэд зориулсан механизмтай байдаг.



Transactions and Concurrency Control

Зэрэгцээ гүйлгээний төрлүүд:

- **Lost Update (Алдагдсан шинэчлэлт)**:
  Хоёр хэрэглэгч нэгэн зэрэг өгөгдлийг шинэчилж, нэгнийх нь шинэчлэлт алдагдах явдал.
- **Dirty Read (Бохир уншилт)**:
  Нэг хэрэглэгчийн хийгээгүй өөрчлөлтийг нөгөө нь унших.
- **Non-repeatable Read (Давтагдахгүй уншилт)**:
  Нэг өгөгдлийг олон удаа уншихад ялгаатай үр дүн буцах.
- **Phantom Read (Сүүдрийн уншилт)**:
  Нэг хэрэглэгч өгөгдлийг уншиж байх үед өөр хэрэглэгч шинэ мөр нэмэхэд ялгаатай үр дүн гарч ирэх.

Transactions and Concurrency Control

Тусгаарлалтын түвшин (Isolation Levels):
Гүйлгээний тусгаарлалтын түвшин нь зэрэгцээ гүйлгээг хэрхэн удирдахыг тодорхойлдог. SQL-д дараах тусгаарлалтын түвшнүүд байдаг:

- **Read Uncommitted**: Бохир уншилтыг зөвшөөрдөг, хамгийн доод түвшин.
- **Read Committed**: Бохир уншилтыг зөвшөөрөхгүй, гэхдээ давтагдахгүй уншилт гарч болзошгүй.
- **Repeatable Read**: Давтагдахгүй уншилтаас сэргийлдэг, гэхдээ сүүдрийн уншилт гарч болзошгүй.
- **Serializable**: Хамгийн өндөр түвшний тусгаарлалт, бүх асуудлаас сэргийлдэг.

SAVEPOINT ашиглах

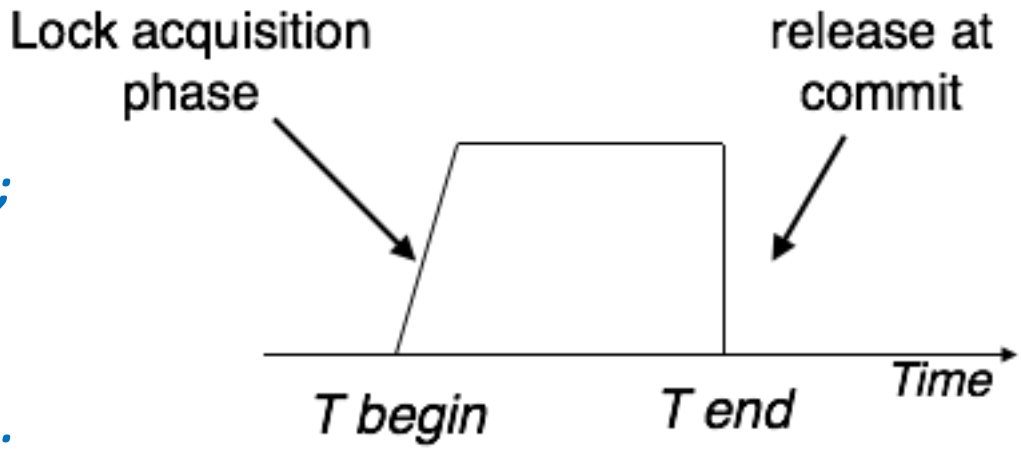
```
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;

BEGIN TRANSACTION;

SELECT balance
FROM accounts
WHERE account_number = 123;

UPDATE accounts
SET balance = balance - 100
WHERE account_number = 123;

COMMIT;
```



- Энэ жишээнд **SERIALIZABLE** түвшнийг тохируулж, гүйлгээний явцад ямар ч зөрчил гарахгүйг баталгаажуулж байна.

Анхаарал тавьсанд баярлалаа

